



Python应用基础

第10章 SQL核心语法

陈刚

中山大学岭南学院

SQL语言核心分类架构

- DDL（数据定义语言）— 定义数据结构
- DML（数据操作语言）— 操作数据内容
- DQL（数据查询语言）— 查询数据（最核心）
- DCL（数据控制语言）— 控制访问权限



定义、修改、删除数据库对象（库、表、索引、视图等）

一、数据定义语言

DDL (Data Definition Language)

1.1 数据库操作

```
-- 创建数据库
CREATE DATABASE SchoolDB;

-- 使用数据库
USE SchoolDB;

-- 删除数据库
DROP DATABASE SchoolDB;
```

你的表是否涉及...	Status使用建议	例子
流程、步骤、生命周期	强烈建议	订单、任务、审批单
逻辑删除（软删除）	非常推荐	用户账号、文章、评论
明确的、有限的几种情形	推荐	客户等级、商品上下架
数据用来记录历史事实，状态单一	不需要	系统操作日志、用户登录记录

1.2 表操作

```
- 创建表（带约束）
CREATE TABLE Students (
  StudentID INT PRIMARY KEY AUTO_INCREMENT, -- 主键+自增
  Name VARCHAR(50) NOT NULL, -- 非空
  Email VARCHAR(100) UNIQUE, -- 唯一
  Age INT CHECK (Age >= 0 AND Age <= 150), -- 检查约束
  Status VARCHAR(20) DEFAULT 'Active' -- 默认值
);

-- 修改表结构
ALTER TABLE Students ADD Phone VARCHAR(20); -- 添加列
ALTER TABLE Students DROP COLUMN Phone; -- 删除列
ALTER TABLE Students MODIFY COLUMN Age SMALLINT; -- 修改列类型
ALTER TABLE Students RENAME COLUMN Status TO State; -- 重命名列

-- 删除表
DROP TABLE Students;

-- 清空表数据（保留结构，不可回滚）
TRUNCATE TABLE TempData;
```

1.3 索引操作

-- 创建索引（加速查询）

```
CREATE INDEX idx_name ON Students (Name);
```

-- 创建唯一索引

```
CREATE UNIQUE INDEX idx_email ON Students (Email);
```

-- 删除索引

```
DROP INDEX idx_name ON Students;
```

索引就像书的目录。如果你想在书里找到某个知识点，有目录可以直接翻到对应页码；没有目录就只能一页一页翻。数据库索引的作用是帮助数据库快速定位数据，避免逐行扫描整张表。

1.4 视图操作

-- 创建视图（虚拟表，简化复杂查询）

```
CREATE VIEW ActiveStudents AS  
SELECT StudentID, Name, Age  
FROM Students  
WHERE Status = 'Active';
```

-- 使用视图

```
SELECT * FROM ActiveStudents;
```

-- 删除视图

```
DROP VIEW ActiveStudents;
```

视图就是一张虚拟表，它不实际存储数据，而是一个保存好的查询。每次你查询视图，数据库都会自动执行视图里定义的那个 SELECT 语句，从原始表中获取最新数据。



对表中的数据进行增、删、改

二、数据操作语言

DML (Data Manipulation Language)

2.1 INSERT（插入数据）

```
-- 插入完整一行
INSERT INTO Students (StudentID, Name, Age, Status)
VALUES (1, '张三', 20, 'Active');

-- 省略列名（需提供所有列的值）
INSERT INTO Students VALUES (2, '李四', 21, 'Active');

-- 批量插入
INSERT INTO Students (StudentID, Name, Age) VALUES
(3, '王五', 22),
(4, '赵六', 19);

-- 复制其他表的数据
INSERT INTO ExcellentStudents (Name, Score)
SELECT Name, Score FROM Scores WHERE Score >= 90;
```

插入就是向数据库的表中添加新数据行。就像在学生花名册上增加一个新同学的信息，写上他的学号、姓名、年龄。

2.2 UPDATE（更新数据）

```
-- 更新所有行（谨慎！）
UPDATE Students SET Status = 'Inactive';

-- 带条件的更新
UPDATE Students
SET Age = Age + 1, Status = 'Active'
WHERE StudentID = 1;

-- 基于其他表更新
UPDATE Students S
JOIN Scores SC ON S.StudentID = SC.StudentID
SET S.Status = 'Excellent'
WHERE SC.Score >= 95;
```

UPDATE 是修改表中已有数据的操作。可理解为在表格上修改已填好的内容，比如发现学生「张三」的年龄写错了，把 20 岁改成 21 岁；或者用户修改自己的登录密码。

2.3 DELETE（删除数据）

-- 删除所有行（可回滚，但慢）

```
DELETE FROM Students;
```

-- 带条件删除

```
DELETE FROM Students WHERE StudentID = 5;
```

-- 基于子查询删除

```
DELETE FROM Students
```

```
WHERE StudentID IN (SELECT StudentID FROM InactiveList);
```

操作	作用	行数变化	常见场景
INSERT	增加新行	增加	注册、下单、发帖
UPDATE	修改已有行	不变	改密码、改状态、改金额
DELETE	删除行	减少	注销账号、取消订单



查询数据，SQL中最核心、最常用的部分

三、数据查询语言

DQL (Data Query Language)

3.1 基础查询

-- 基本结构

SELECT 列名1, 列名2...

FROM 表名

WHERE 条件

GROUP BY 分组列

HAVING 分组后条件

ORDER BY 排序列

LIMIT 行数;

-- 例子: 查询所有学生

SELECT * FROM Students;

-- 查询特定列并去重

SELECT DISTINCT Class FROM Students;

-- 列别名

SELECT Name AS 姓名, Age AS 年龄 FROM Students;

3.2 条件筛选 (WHERE)

-- 比较运算符

SELECT * FROM Students WHERE Age > 18;

SELECT * FROM Students WHERE Age = 20;

SELECT * FROM Students WHERE Age BETWEEN 18 AND 22;

-- 逻辑运算符

SELECT * FROM Students WHERE Age > 18 AND Class = 'A';

SELECT * FROM Students WHERE Class = 'A' OR Class = 'B';

SELECT * FROM Students WHERE NOT Status = 'Inactive';

-- IN 操作符

SELECT * FROM Students WHERE Class IN ('A', 'B', 'C');

-- 模糊查询 LIKE

SELECT * FROM Students WHERE Name LIKE '张%'; -- 以张开头

SELECT * FROM Students WHERE Name LIKE '_三'; -- 第二个字是"三"且

-- NULL 值处理 (重要!)

SELECT * FROM Students WHERE Email IS NULL;

SELECT * FROM Students WHERE Email IS NOT NULL;

3.3 聚合函数

-- 常用聚合函数

```
SELECT
  COUNT(*) AS 总人数,           -- 总行数
  COUNT(DISTINCT Class) AS 班级数, -- 去重统计
  AVG(Age) AS 平均年龄,         -- 平均值
  SUM(Score) AS 总分,           -- 求和
  MAX(Score) AS 最高分,         -- 最大值
  MIN(Score) AS 最低分          -- 最小值
FROM Scores;
```

聚合函数是对一组数据（多行）进行计算，返回单个结果值的函数。

函数	作用	结果	类比
COUNT()	数数	一个数字	"全班有多少人"
SUM()	求和	一个数字	"总分是多少"
AVG()	求平均	一个数字	"平均分是多少"
MAX()	找最大	一个值	"谁分最高"
MIN()	找最小	一个值	"谁分最低"

3.4 分组查询 (GROUP BY + HAVING)

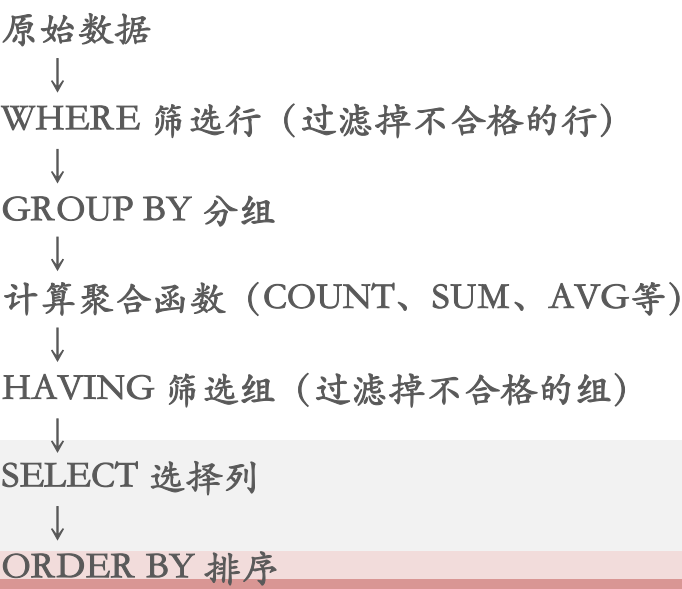
```
- 统计每个班级的人数
SELECT Class, COUNT(*) AS 人数
FROM Students
GROUP BY Class;

-- 统计每个班级的平均分（限定80分以上才显示）
SELECT Class, AVG(Score) AS AvgScore
FROM Scores
GROUP BY Class
HAVING AvgScore > 80;

-- 重要规则：WHERE vs HAVING
-- WHERE 在分组前筛选（对行）
-- HAVING 在分组后筛选（对组）
SELECT Class, AVG(Score) AS AvgScore
FROM Scores
WHERE Score >= 60           -- 先过滤及格的学生
GROUP BY Class
HAVING COUNT(*) >= 5;      -- 再筛选人数≥5的班级
```

GROUP BY 和 HAVING 是 SQL 中用于分组统计的两个关键字，通常配合聚合函数（COUNT、SUM、AVG 等）一起使用。

关键字	作用	类比	执行时机
GROUP BY	把数据按某列分组	把全班学生按班级分成几个组	WHERE 之后
HAVING	对分组后的结果进行筛选	只保留人数大于 30 人的班级	GROUP BY 之后



3.5 排序（ORDER BY）

```
-- 单列排序
SELECT * FROM Students ORDER BY Age DESC; -- 降序
SELECT * FROM Students ORDER BY Age ASC;  -- 升序（默认）

-- 多列排序（先按班级升序，再按年龄降序）
SELECT * FROM Students ORDER BY Class ASC, Age DESC;
```

3.6 限制结果（LIMIT）

```
-- 返回前5行
SELECT * FROM Students LIMIT 5;

-- 分页查询（跳过前5行，取10行）
SELECT * FROM Students LIMIT 5, 10;

-- 最常见的用法：找前三名
SELECT Name, Score FROM Scores
ORDER BY Score DESC LIMIT 3;
```

3.7 表连接 (JOIN)

-- 准备示例数据

-- Students表: StudentID, Name, Class

-- Scores表: StudentID, Course, Score

-- INNER JOIN (只返回匹配的行)

```
SELECT Students.Name, Scores.Score
```

```
FROM Students
```

```
INNER JOIN Scores ON Students.StudentID = Scores.StudentID;
```

-- LEFT JOIN (左表全部, 右表匹配不到填NULL)

```
SELECT Students.Name, Scores.Score
```

```
FROM Students
```

```
LEFT JOIN Scores ON Students.StudentID = Scores.StudentID;
```

-- RIGHT JOIN (右表全部, 左表匹配不到填NULL)

```
SELECT Students.Name, Scores.Score
```

```
FROM Students
```

```
RIGHT JOIN Scores ON Students.StudentID = Scores.StudentID;
```

-- 自连接 (同一张表当两张表用)

-- 查询员工及其上级

```
SELECT e.Name AS Employee, m.Name AS Manager
```

```
FROM Employees e
```

```
LEFT JOIN Employees m ON e.ManagerID = m.EmployeeID;
```

-- 多表连接

```
SELECT Students.Name, Scores.Course, Scores.Score, Classes.Room
```

```
FROM Students
```

```
JOIN Scores ON Students.StudentID = Scores.StudentID
```

```
JOIN Classes ON Students.ClassID = Classes.ClassID;
```

JOIN 是用来把多张表的数据「连在一起」查询的操作。现实世界的数据是分散存储的, 例如学生信息在一张表, 成绩在另一张表。要同时看到「张三」和他的「数学分数」, 就必须用 JOIN 把两张表连接起来。

JOIN 类型	返回结果
INNER JOIN	两表都匹配的行
LEFT JOIN	左表全部 + 右表匹配的 (无则NULL)
RIGHT JOIN	右表全部 + 左表匹配的 (无则NULL)
FULL JOIN	两表全部, 不匹配的填NULL
CROSS JOIN	两表的笛卡尔积 (所有组合)
SELF JOIN	自己连接自己

Student表		Score表		INNER JOIN 结果		LEFT JOIN 结果		RIGHT JOIN 结果		FULL JOIN 结果	
ID	Name	ID	Score	Name	Score	Name	Score	Name	Score	Name	Score
1	张三	1	90	张三	90	张三	90	张三	90	张三	90
2	李四	2	80	李四	80	李四	80	李四	80	李四	80
3	王五	4	95			王五	NULL	NULL	95	王五	NULL
										NULL	95

3.8 合并查询 (UNION)

```
-- UNION (自动去重)
SELECT Name FROM Students_A
UNION
SELECT Name FROM Students_B;

-- UNION ALL (不去重, 效率更高)
SELECT Name FROM Students_A
UNION ALL
SELECT Name FROM Students_B;

-- 示例数据
-- Students_A: 张三, 李四, 王五
-- Students_B: 李四, 赵六, 李四

-- UNION 结果: 张三, 李四, 王五, 赵六
-- UNION ALL 结果: 张三, 李四, 王五, 李四, 赵六, 李四
```

JOIN: 把两张表左右拼在一起, 列数变多 (像拼接两张Excel表)

UNION: 把两张表上下叠在一起, 行数变多 (像把两张表的内容复制粘贴到同一个工作表里)

3.9 子查询

```
-- WHERE子查询
SELECT Name, Score
FROM Scores
WHERE Score > (SELECT AVG(Score) FROM Scores);

-- FROM子查询 (将查询结果当作临时表)
SELECT AVG(Age)
FROM (SELECT * FROM Students WHERE Class = 'A') AS ClassAStudents;

-- EXISTS子查询
SELECT Name
FROM Students S
WHERE EXISTS (SELECT 1 FROM Scores SC WHERE SC.StudentID = S.StudentID);
```

3.10 执行顺序（理解查询的关键）

书写顺序: SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

执行顺序: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

示例说明为什么WHERE中不能用别名:

```
SELECT Name AS 学生姓名, Age AS 年龄
```

```
FROM Students
```

```
WHERE 学生姓名 = '张三';  -- 错误! WHERE执行时别名还未生成
```




控制数据库的访问权限

四、数据控制语言

DCL (Data Control Language)

4.1 创建用户

```
-- MySQL语法
CREATE USER 'student_user'@'localhost' IDENTIFIED BY
'password123';

-- SQL Server语法
CREATE USER student_user FOR LOGIN student_login;
```

4.2 授予权限 (GRANT)

```
-- 授予查询权限
GRANT SELECT ON SchoolDB.Students TO 'student_user'@
'localhost';

-- 授予增删改查权限
GRANT SELECT, INSERT, UPDATE, DELETE ON SchoolD
B.* TO 'student_user'@'localhost';

-- 授予所有权限
GRANT ALL PRIVILEGES ON SchoolDB.* TO 'student_user
'@'localhost';

-- 允许授权给其他用户
GRANT SELECT ON SchoolDB.Students TO 'student_user'@
'localhost' WITH GRANT OPTION;
```

命令	作用	类比
CREATE USER	创建数据库账户	给大楼发门禁卡
GRANT	赋予账户操作权限	设置这张卡能开哪几扇门（能进哪个库、哪个表）
REVOKE	收回权限	取消门禁权限

CREATE USER 和 GRANT 是 SQL 中用于权限管理的命令，用来控制谁能访问数据库、能做什么操作。

基本语法

GRANT 权限列表 ON 数据库对象 TO 用户名;

数据库对象	写法	说明
全局	*,*	所有数据库的所有表
数据库级	SchoolDB.*	指定数据库的所有表
表级	SchoolDB.Students	指定数据库的指定表
列级	(Name, Age) ON Students	只限特定列（较少用）

权限	作用	对应 SQL
SELECT	查询数据	SELECT ...
INSERT	插入数据	INSERT INTO ...
UPDATE	更新数据	UPDATE ...
DELETE	删除数据	DELETE FROM ...
CREATE	创建表/数据库	CREATE TABLE ...
DROP	删除表/数据库	DROP TABLE ...
INDEX	创建/删除索引	CREATE INDEX ...
ALTER	修改表结构	ALTER TABLE ...
ALL PRIVILEGES	除授权外的所有权限	—
WITH GRANT OPTION	允许把权限再授权给别人	—

4.3 撤销权限 (REVOKE)

-- 撤销查询权限

```
REVOKE SELECT ON SchoolDB.Students FROM 'student_  
user'@'localhost';
```

-- 撤销所有权限

```
REVOKE ALL PRIVILEGES ON SchoolDB.* FROM 'stude  
nt_user'@'localhost';
```

4.4 查看权限

-- MySQL查看用户权限

```
SHOW GRANTS FOR 'student_user'@'localhost';
```



快速参考卡片

-- DDL

CREATE DATABASE db;
CREATE TABLE t (...);
ALTER TABLE t ADD col;
CREATE INDEX idx ON t(col);

DROP DATABASE db;
DROP TABLE t;
TRUNCATE TABLE t;
CREATE VIEW v AS SELECT...

-- DML

INSERT INTO t VALUES(...);
DELETE FROM t WHERE...;

UPDATE t SET col=val WHERE...;

-- DQL

SELECT cols FROM t
WHERE condition
GROUP BY col
HAVING condition
ORDER BY col
LIMIT n;

-- DCL

CREATE USER 'user'@'host' IDENTIFIED BY 'pwd';
GRANT SELECT ON db.* TO 'user'@'host';
REVOKE SELECT ON db.* FROM 'user'@'host';
SHOW GRANTS FOR 'user'@'host';